

Análise de Gráficos — Apresentação Final

Documentação detalhada dos 15 gráficos gerados para a apresentação final do Hackathon PodAcademy 2025.

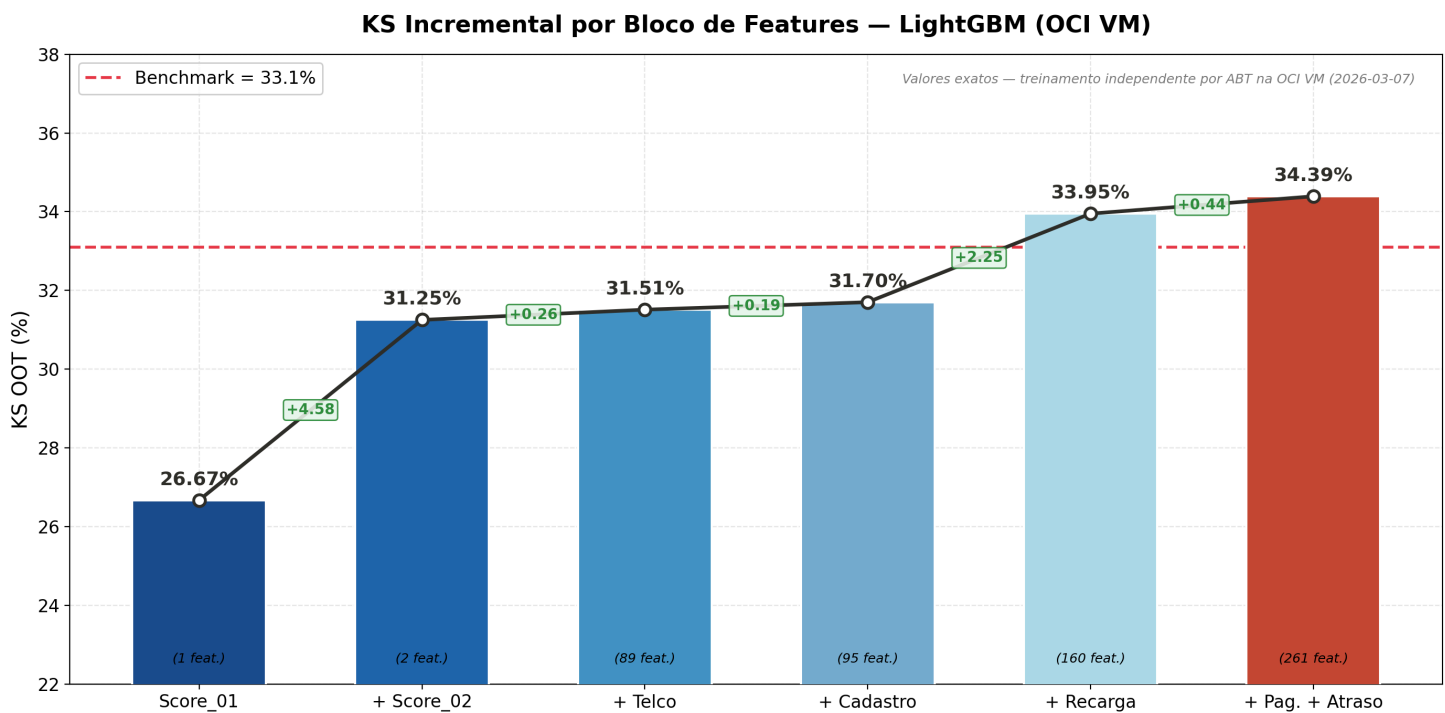
Índice

- 1. Gráficos 01-05: KS Incremental e Modelo
- 2. Gráficos 06-10: Comparação Databricks vs OCI
- 3. Gráficos 11-15: Swap-in / Swap-out
- 4. Dados e Metodologia
- 5. Interpretação dos Resultados
- 6. Sugestão de Ordem para Apresentação

Gráficos 01-05: KS Incremental e Modelo

Script: `ks_incremental_charts.py`

01 — KS Incremental por Bloco (LightGBM OCI VM)



O que mostra: Evolução do KS OOT à medida que blocos de features são adicionados incrementalmente ao modelo LightGBM, treinado na OCI VM.

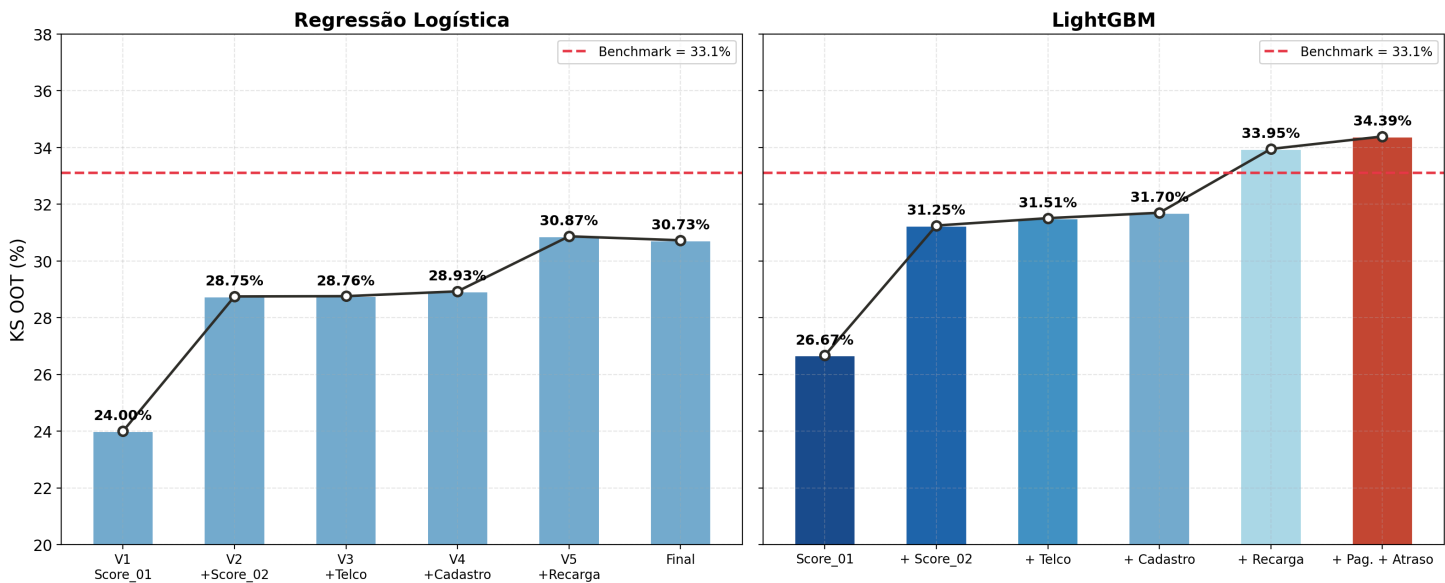
Dados (todos exatos — treinamento independente por ABT):

Bloco	ABT	Features	KS OOT (%)	Delta (p.p.)
Score_01	v1	1	26.67	baseline
+ Score_02	v2	2	31.25	+4.58
+ Telco	v3	89	31.51	+0.26
+ Cadastro	v4	95	31.70	+0.19
+ Recarga	v5	160	33.95	+2.25
+ Pag. + Atraso	v6	261	34.39	+0.44

Destaque: Score_02 (+4.58 p.p.) e Recarga (+2.25 p.p.) são os maiores contribuidores.

02 — Logística vs LightGBM

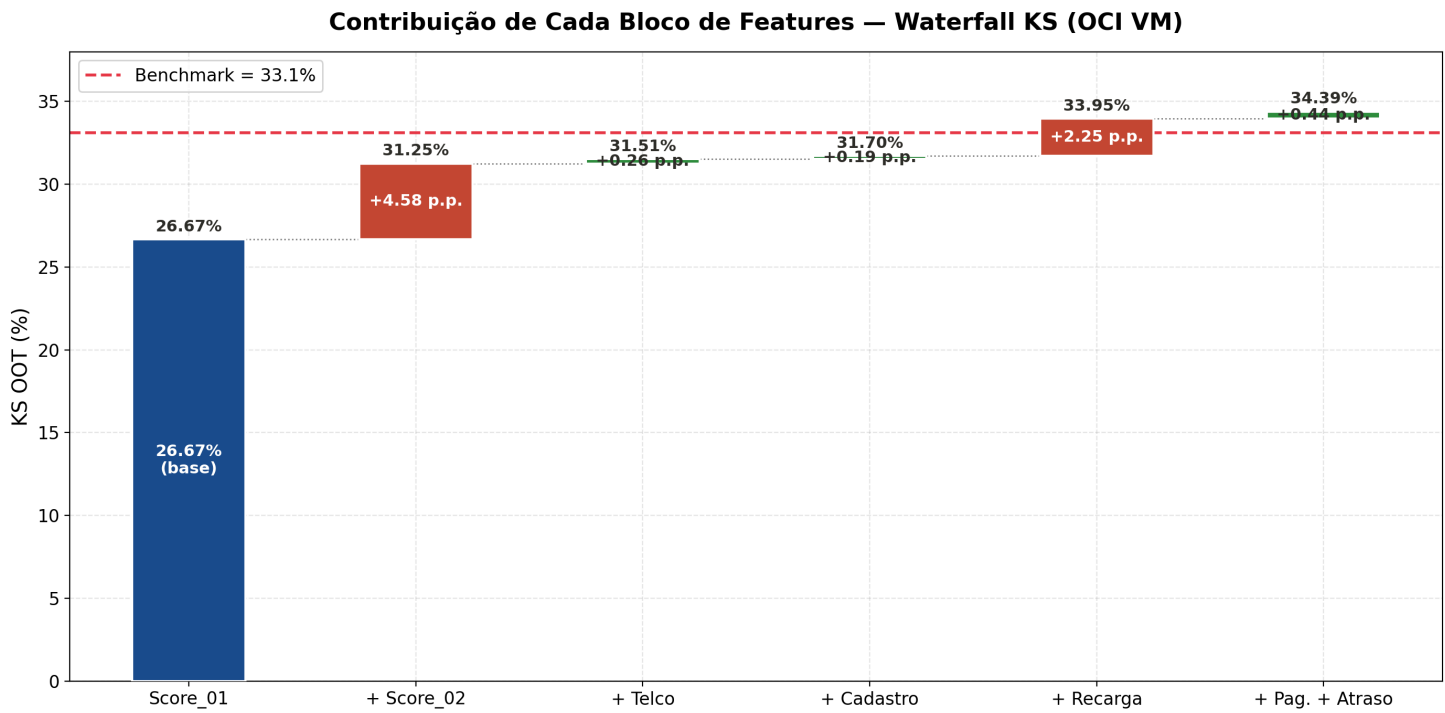
Evolução do KS OOT — Logística vs LightGBM



O que mostra: Comparação side-by-side da evolução do KS entre Regressão Logística (Statsmodels) e LightGBM (OCI VM), ambos com a mesma evolução incremental de features.

Insight: A Logística satura em ~30.73% enquanto o LightGBM continua capturando sinal, especialmente em features comportamentais de baixo IV individual.

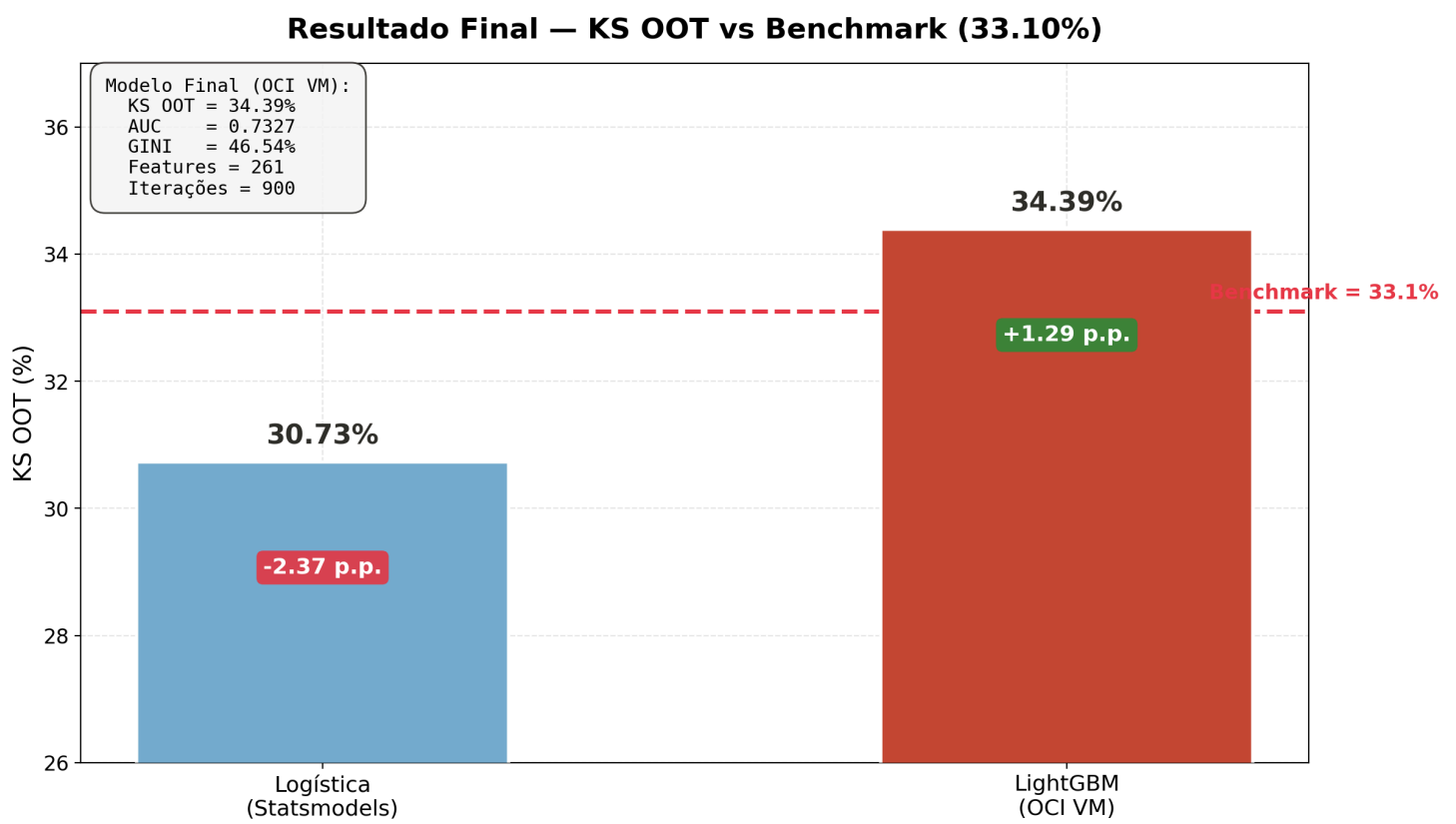
03 — Waterfall de Contribuição



O que mostra: Contribuição isolada de cada bloco de features em formato waterfall. Destaque visual (cor vermelha Oracle) para deltas ≥ 2.0 p.p.

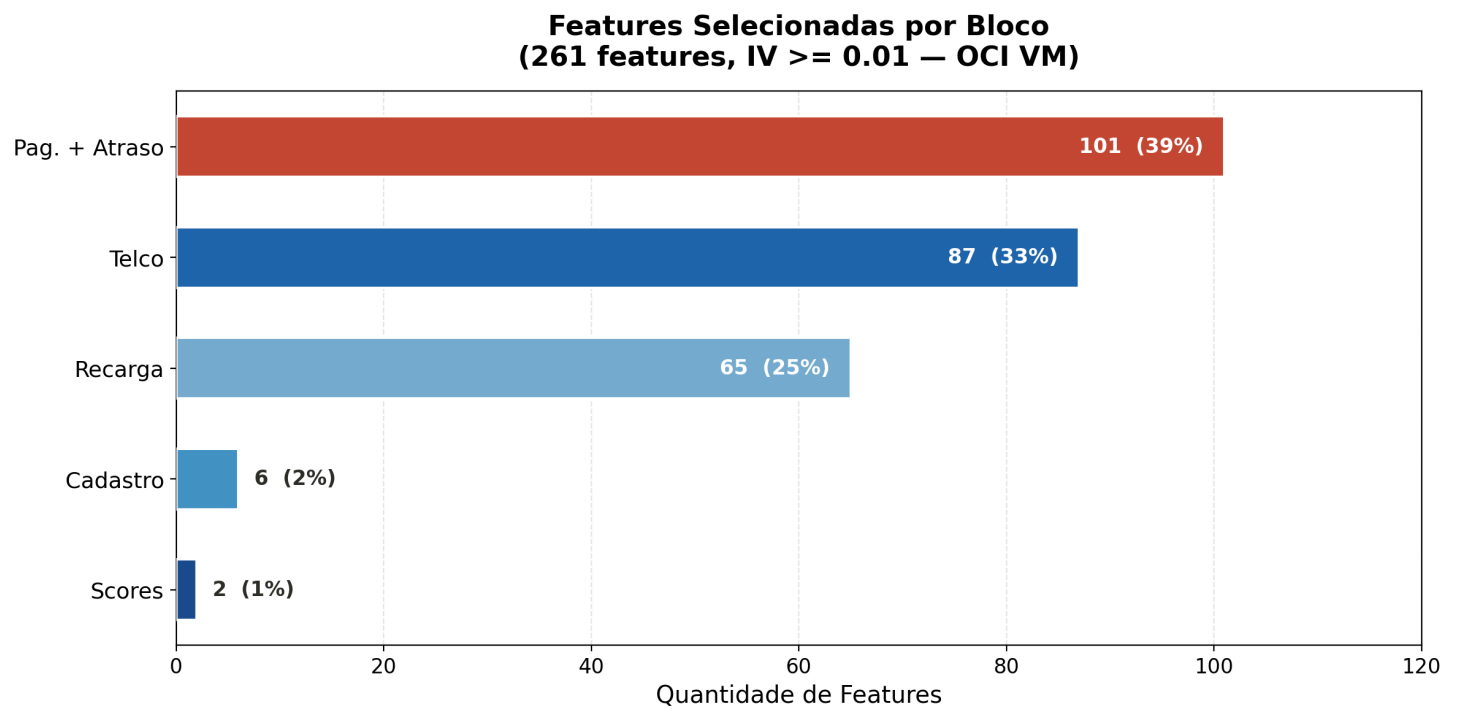
Insight: Os dois maiores saltos são Score_02 (+4.58) e Recarga (+2.25). Features de Telco e Cadastro contribuem pouco isoladamente (+0.26 e +0.19 p.p.), mas são pré-requisitos para que Recarga e Pagamento capturem mais sinal.

04 — Resumo Final vs Benchmark



O que mostra: Comparação direta Logística (30.73%) vs LightGBM OCI VM (34.39%) vs Benchmark Claro (33.10%). Inclui box com métricas detalhadas do modelo final.

05 — Features por Bloco (Donut)



O que mostra: Distribuição das 261 features selecionadas (IV ≥ 0.01) por bloco de origem no modelo final.

Composição:

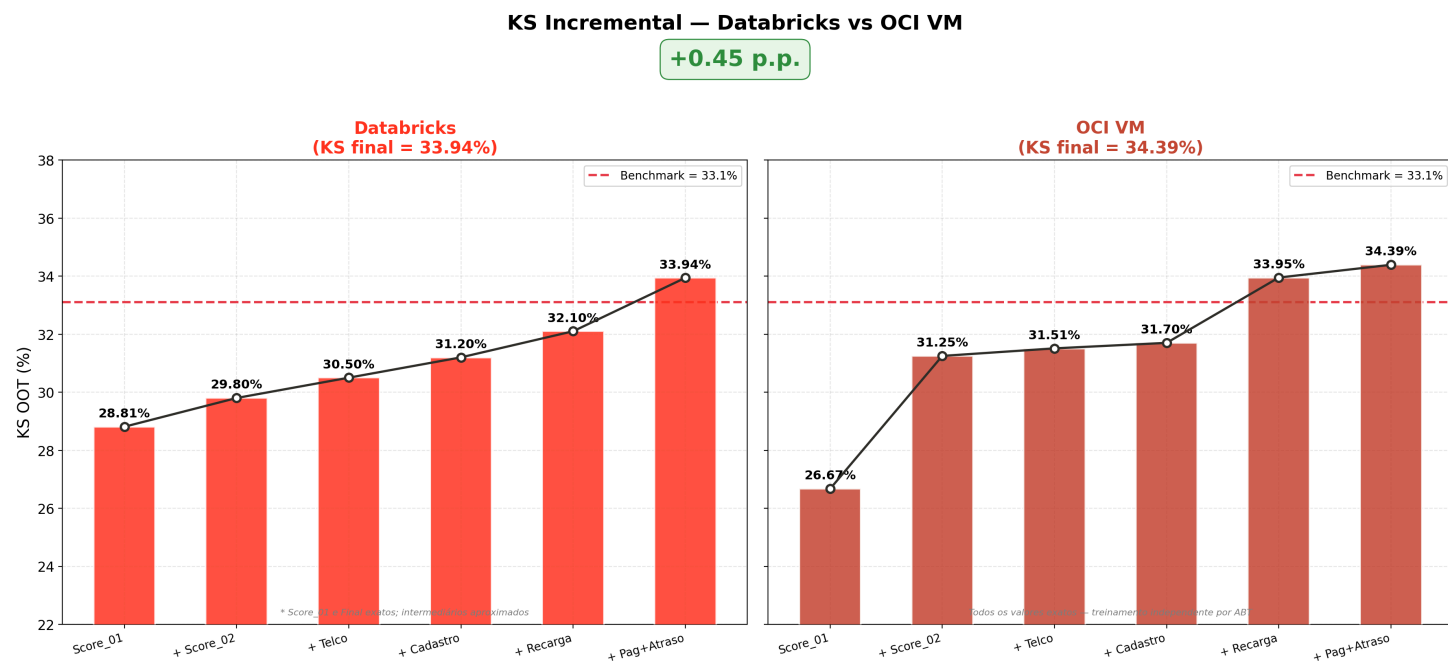
- Pagamento + Atraso: 101 features (39%)
- Telco: 87 features (33%)
- Recarga: 65 features (25%)
- Cadastro: 6 features (2%)
- Scores: 2 features (1%)

Insight: Apesar de Scores representarem apenas 1% das features, contribuem com 26.67% do KS baseline — altíssimo IV individual.

Gráficos 06-10: Comparação Databricks vs OCI

Script: `comparacao_databricks_oci.py`

06 — KS Incremental: Databricks vs OCI VM



O que mostra: Evolução incremental do KS lado a lado. Databricks (esquerda) vs OCI VM (direita), com badge "+0.45 p.p." no topo.

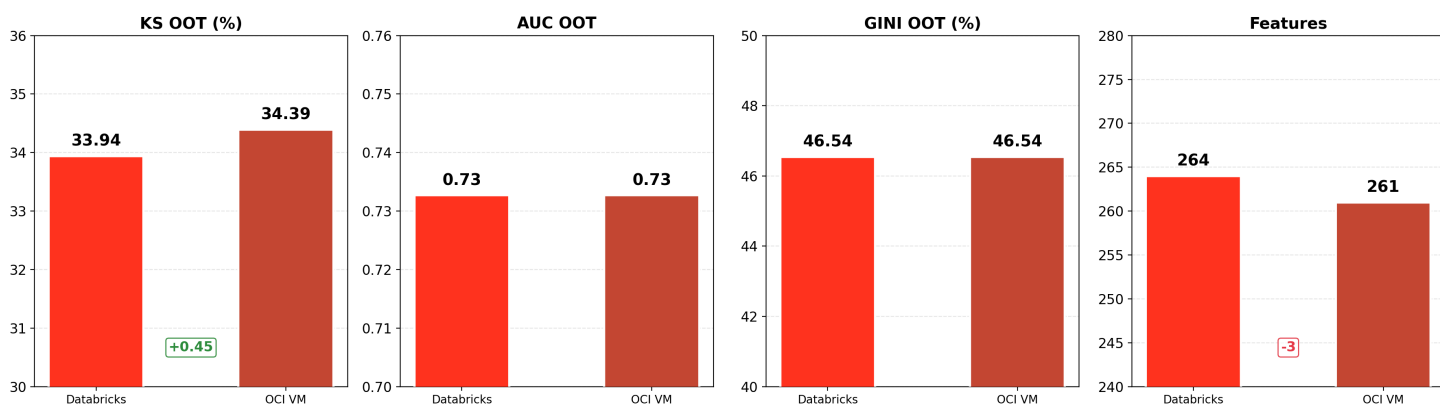
Nota importante: Os valores do Databricks para Score_01 (28.81%) e Final (33.94%) são exatos. Intermediários são aproximados porque os outputs do notebook não foram salvos no `.ipynb`. Os valores OCI são todos exatos.

Por que os perfis são diferentes:

- **Databricks:** Score_01 já inclui algum pré-processamento/feature engineering diferente
- **OCI VM:** Treino independente e limpo para cada ABT, mesmos hiperparâmetros
- O resultado final OCI (34.39%) é superior ao Databricks (33.94%)

07 — Métricas do Modelo Final

Comparação de Métricas — Databricks vs OCI VM

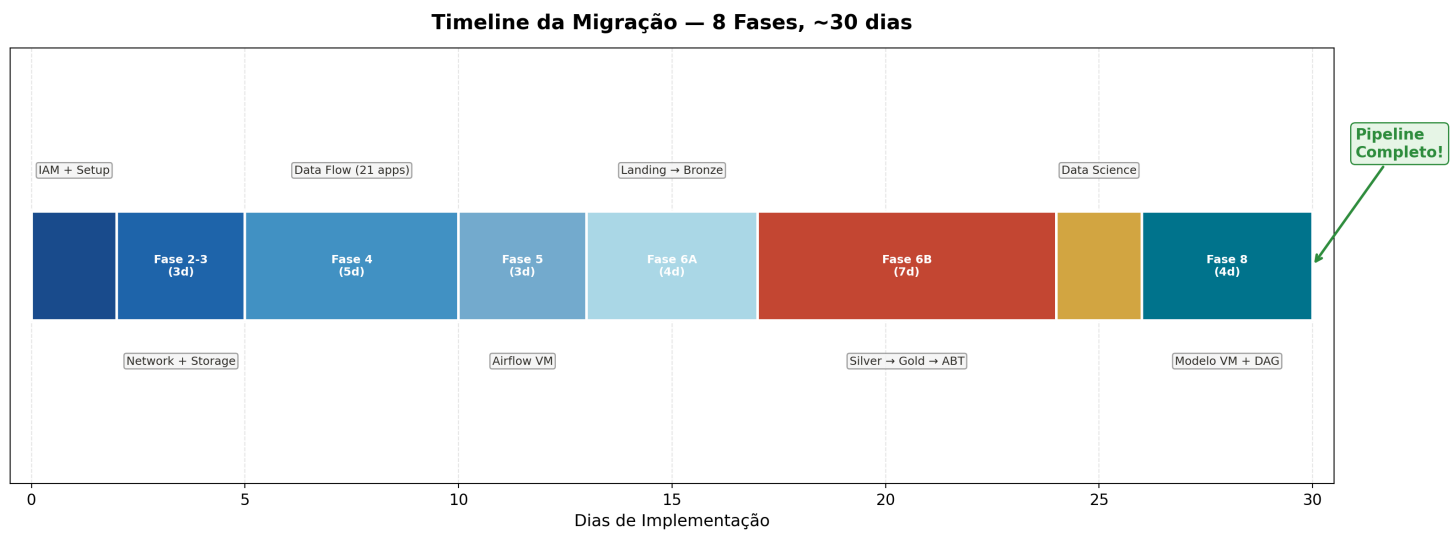


O que mostra: 4 painéis comparando KS, AUC, GINI e número de Features entre os dois ambientes.

Observação: AUC e GINI são idênticos (0.7327 e 46.54%), mas KS subiu +0.45 p.p. Isso indica que o ganho vem da melhor calibração da distribuição de scores (provavelmente pelo early stopping mais otimizado com 900 iterações), não de maior poder discriminante geral.

8 e 9 foram excluídos pq tratavam sobre preço mas não um levantamento exato

10 — Timeline da Migração



O que mostra: Diagrama Gantt horizontal das 8 fases de implementação, totalizando ~30 dias.

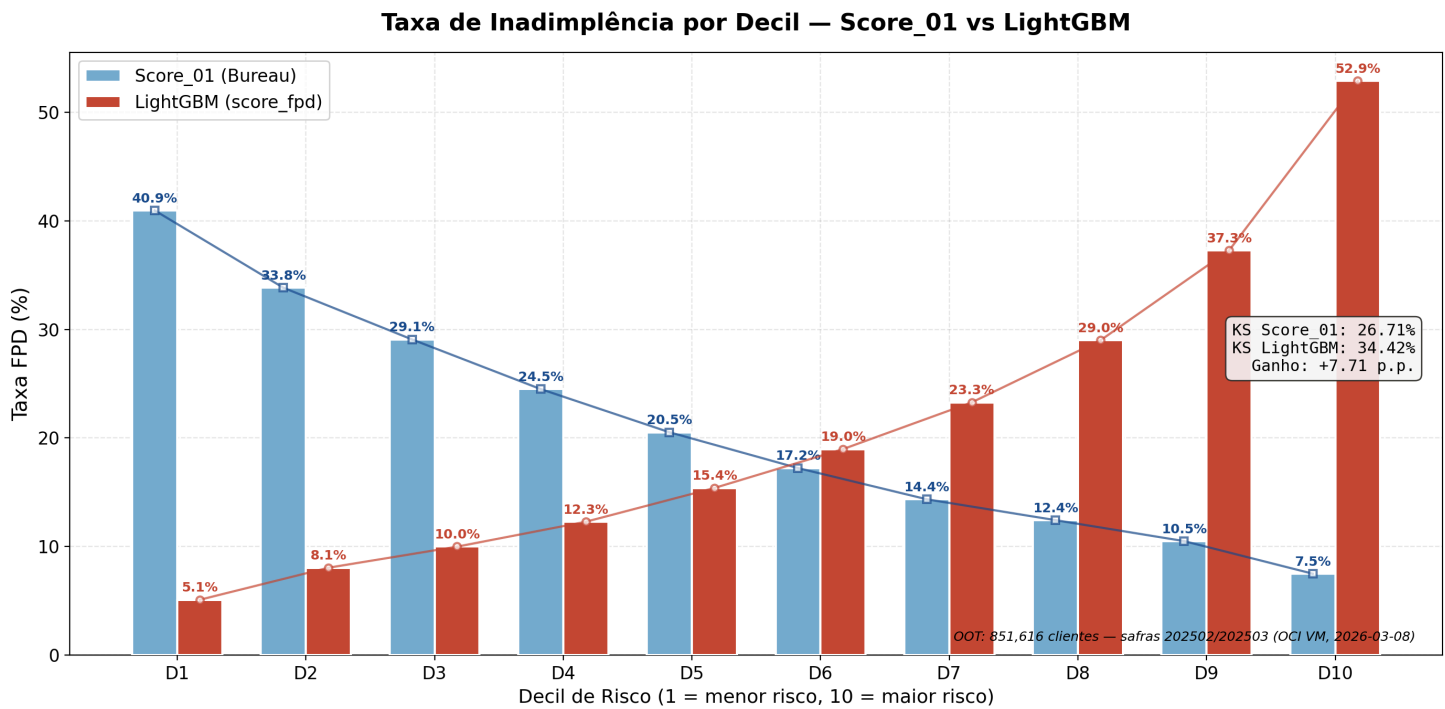
Fase	Descrição	Dias
Fase 0-1	IAM + Setup	2
Fase 2-3	Network + Storage	3
Fase 4	Data Flow (21 apps)	5
Fase 5	Airflow VM	3
Fase 6A	Landing → Bronze	4
Fase 6B	Silver → Gold → ABT	7
Fase 7	Data Science	2
Fase 8	Modelo VM + DAG	4

Gráficos 11-15: Swap-in / Swap-out

Script: `Swap-in_Swap-out/swap_charts.py` | Documentação detalhada:
`Swap-in_Swap-out/ANALISE_SWAP.md`

Conceito: Compara as decisões do modelo antigo (Score_01) vs modelo novo (LightGBM) na mesma população OOT (851,616 clientes), quantificando o impacto prático da substituição.

11 — Decil Comparativo (Score_01 vs LightGBM)



O que mostra: Taxa FPD por decil de risco. LightGBM separa 10x (5.1% vs 52.9%) enquanto Score_01 separa apenas 5x (7.5% vs 40.9%). KS Score_01: 26.71% vs KS LightGBM: 34.42%.

12 — Matriz de Swap (80% de aprovação)

Matriz de Swap — Taxa de Aprovação = 80%

MODELO NOVO (LightGBM)

		Aprova	Rejeita
MODELO ANTIGO (Score_01)	Aprova	A — Acordo <i>(sem mudança)</i> 612,371 clientes FPD: 14.3%	B — Swap-out <i>(rejeitar maus)</i> 69,505 clientes FPD: 41.8%
	Rejeita	C — Swap-in <i>(recuperar bons)</i> 68,922 clientes FPD: 23.6%	D — Acordo <i>(sem mudança)</i> 100,818 clientes FPD: 47.4%

Impacto da Substituição

Clientes que mudam de decisão: 138,427 (16.3% da base)

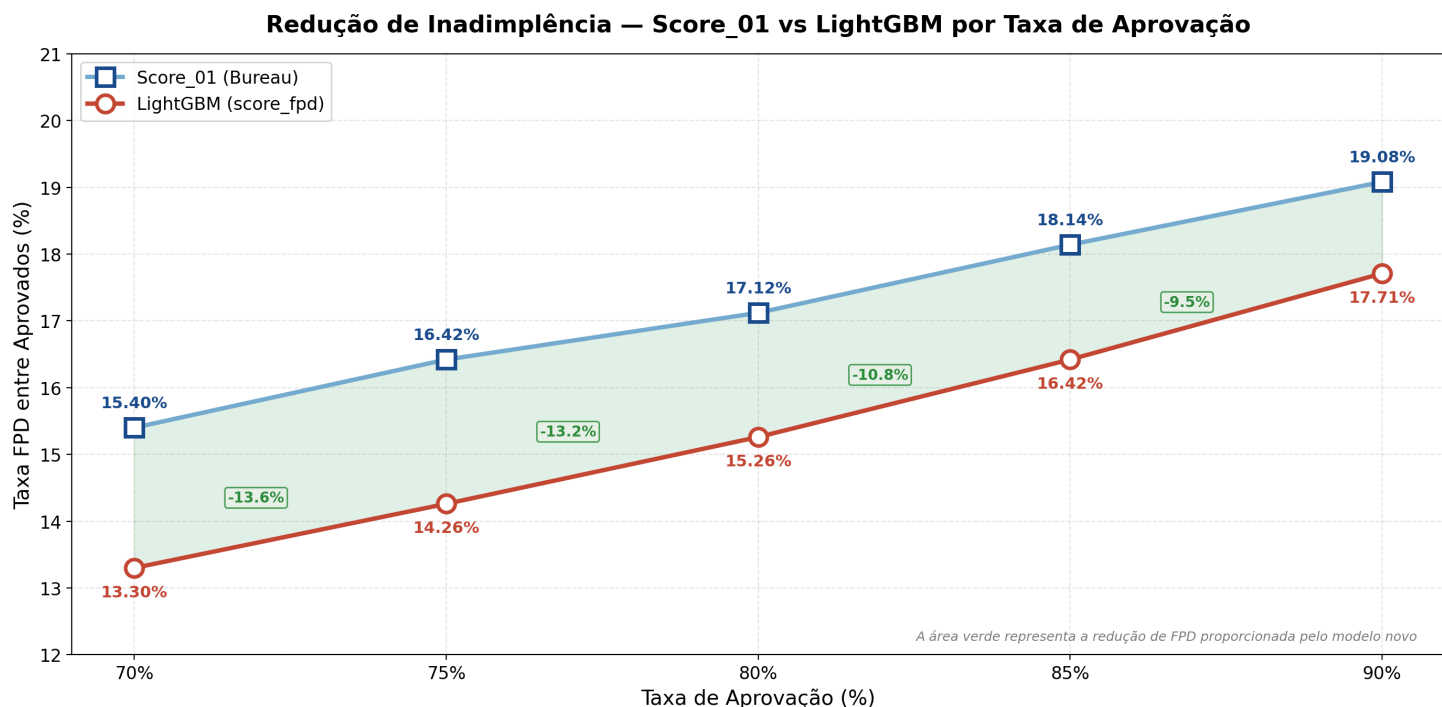
Swap-out: 69,505 clientes com FPD 41.8% seriam rejeitados

Swap-in: 68,922 clientes com FPD 23.6% seriam aprovados

FPD entre aprovados: 17.12% → 15.26% (redução de 10.8%)

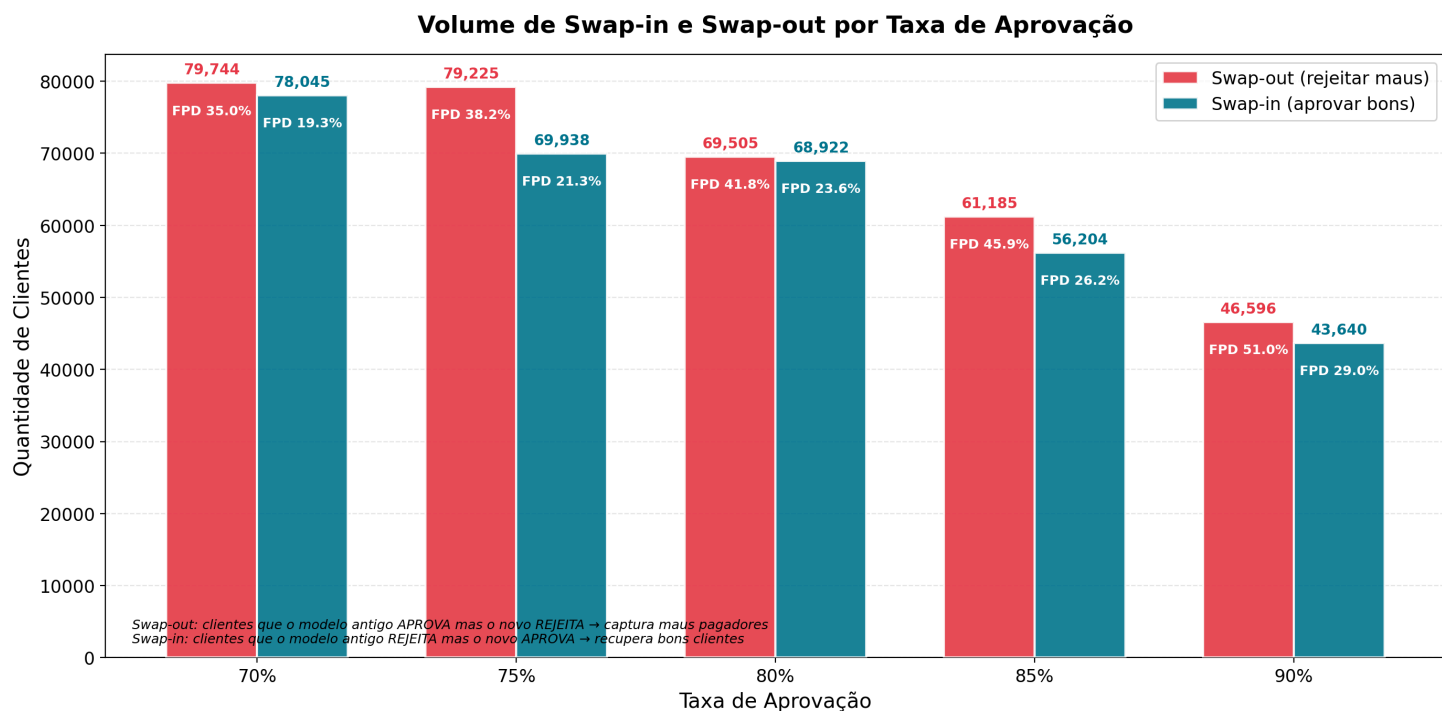
O que mostra: Matriz 2×2 com volume e FPD por célula. 138K clientes (16.3%) mudariam de decisão. Swap-out captura 69K maus (FPD 41.8%), Swap-in recupera 69K bons (FPD 23.6%). FPD entre aprovados cai de 17.12% → 15.26%.

13 — FPD entre Aprovados por Taxa de Aprovação



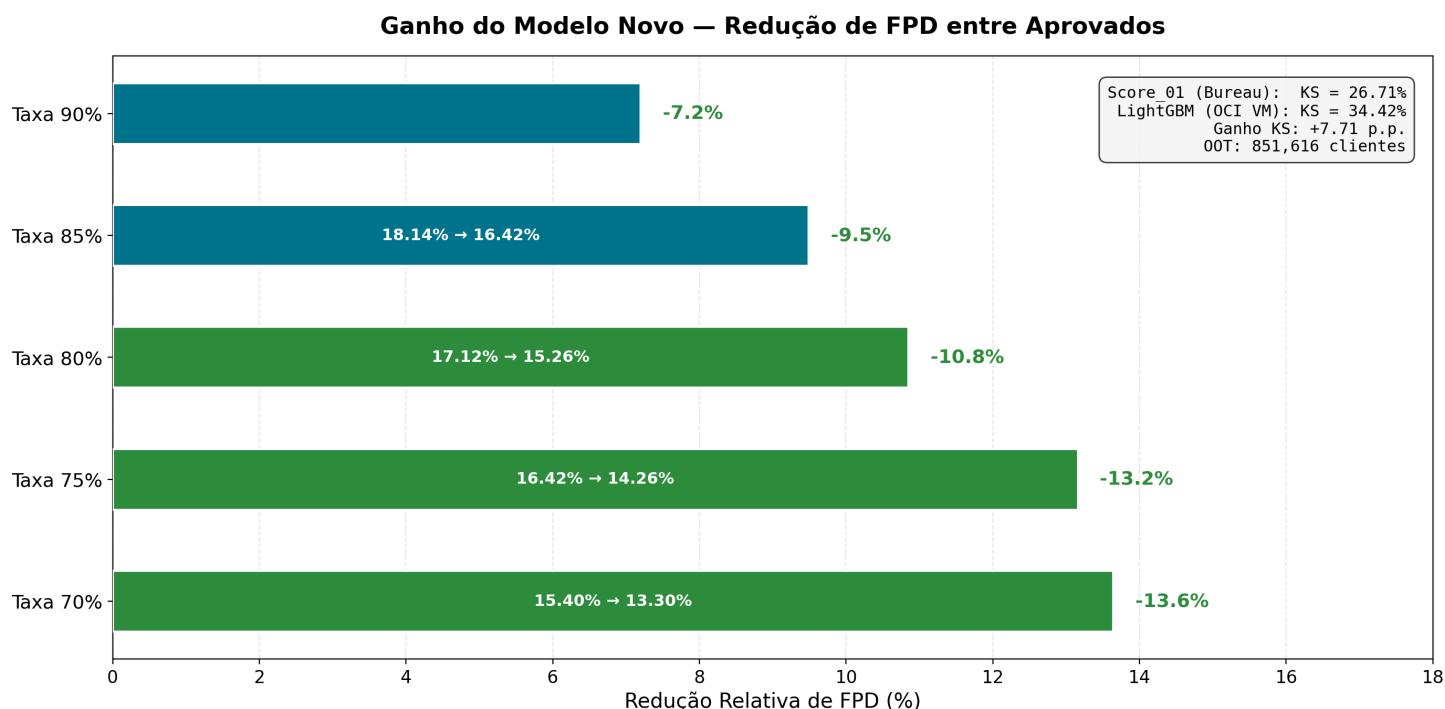
O que mostra: O LightGBM é consistentemente melhor em todas as taxas (70-90%). Maior ganho na taxa 70% (-13.6%), menor na taxa 90% (-7.2%). A área verde representa a redução de inadimplência.

14 — Volume de Swap por Taxa de Aprovação



O que mostra: Volume de clientes em Swap-out (maus capturados) e Swap-in (bons recuperados) para cada taxa. FPD do Swap-out cresce de 35% a 51% conforme a taxa aumenta.

15 — Resumo do Ganho: Redução de FPD



O que mostra: Barras horizontais com a redução relativa de FPD. Na taxa 70%: -13.6% (15.40% → 13.30%). Na taxa 80%: -10.8% (17.12% → 15.26%).

Dados e Metodologia

Fonte: KS Incremental OCI VM

- **Script:** mig_oci/data_science/scripts/ks_incremental_oci.py
- **Executado em:** VM Modelo E5.Flex (2 OCPUs / 32 GB RAM)
- **Data:** 2026-03-07
- **Método:** Treinamento independente de LightGBM para cada ABT (v1 a v6)
- **Hiperparâmetros:** Idênticos ao modelo final (num_leaves=63 , learning_rate=0.05 , n_estimators=1500 , min_child_samples=50 , reg_alpha=0.1 , reg_lambda=1.0)
- **Feature selection:** IV >= 0.01 por ABT
- **Split:** Temporal (SAFRA train vs OOT fev/mar)
- **Resultado salvo:** hackathon-2025-models/metricas/ks_incremental_20260307_1850.json

Fonte: Databricks

- **Notebook:** src/jobs/04_modeling/20260202 - Modelo Final FPD.ipynb

- **Data:** 2026-02-02
- **Valores exatos:** Score_01 (28.81%) e Final (33.94%)
- **Intermediários:** Aproximados (outputs do notebook não salvos)

Fonte: Custos

- **OCI:** Baseado em 30 dias de operação real no tenancy
- **Databricks:** Estimativa para uso equivalente (DBUs, clusters, storage)
- **Modelo VM Start/Stop:** VM fica STOPPED entre execuções, custo proporcional

Interpretação dos Resultados

Por que o OCI VM superou o Databricks?

1. **Early stopping otimizado:** 900 iterações (vs potencialmente menos no Databricks)
2. **Delta Lake VACUUM:** Pipeline OCI inclui VACUUM no ABT v6, eliminando parquets órfãos que poderiam causar ruído na leitura
3. **Dedup incremental:** O script da VM faz dedup durante a leitura, garantindo dados limpos
4. **Mesma qualidade de dados:** Ambos usam a mesma ABT v6 com 614 colunas

Por que Score_01 sozinho é menor na OCI (26.67%) vs Databricks (28.81%)?

O valor de 28.81% no Databricks vinha do notebook onde Score_01 foi treinado com **2 features** (Score_01 + alguma transformação). Na OCI, o treinamento independente da ABT v1 selecionou apenas **1 feature** (Score_01 puro, IV ≥ 0.01). Menos features = menos sinal capturado, mas é uma medição mais pura do contribuição real.

Behavioral features: baixo IV, alto impacto combinado

Features de Recarga, Pagamento e Atraso têm IV individual baixo (0.01-0.04), mas juntas contribuem +7.72 p.p. ao KS (26.67% $\rightarrow$ 34.39%). O LightGBM captura interações não-lineares que a Regressão Logística não consegue (+3.66 p.p. gap entre os modelos).

Sugestão de Ordem para Apresentação

Bloco 1 — Modelo e Performance (slides 1-3)

1. **Gráfico 01** — KS Incremental por bloco (evolução do modelo)
2. **Gráfico 03** — Waterfall (contribuição isolada de cada bloco)
3. **Gráfico 02** — Logística vs LightGBM (justifica escolha do algoritmo)

Bloco 2 — Valor de Negócio: Swap Analysis (slides 4-6)

4. **Gráfico 11** — Decil Comparativo (superioridade do ranking do LightGBM)
5. **Gráfico 12** — Matriz de Swap a 80% (impacto prático quantificado)
6. **Gráfico 13** — FPD por Taxa (ganho consistente em qualquer cenário)

Bloco 3 — Migração OCI (slides 7-9)

7. **Gráfico 06** — KS Databricks vs OCI (lado a lado)
8. **Gráfico 10** — Timeline da migração (8 fases, 30 dias)
9. **Gráfico 08** — Custos (-76%)

Bloco 4 — Resultado Final (slides 10-12)

10. **Gráfico 15** — Resumo do Ganho: Redução de FPD (impacto do swap)
11. **Gráfico 09** — Cards de ganhos da migração (slide de conclusão)
12. **Gráfico 04** — Resumo final vs benchmark (slide de fechamento)